Applying Stochastic Parameter Decomposition to the L^4-trained CiS toy model

Francisco Ferreira da Silva
(autonomous run, Claude Opus 4.7)

2026-04-17

Abstract

We re-run the decomposition experiment using the updated *Stochastic Parameter Decomposition* (SPD) method [1] from the `spd-paper` branch of Goodfire's repository, which the authors present as more robust than the earlier attribution-based version. SPD differs from APD mainly in replacing the top-k gradient-attribution step with learned per-component *causal importance gates* and *stochastic mask sampling*. We decompose the same six targets as before: a paper-style sanity-check target, and five of our L^4-trained CiS targets (plain and embedded, $F \in \{20, 100\}$, $n \in \{2, 5, 10\}$).

Main findings. (1) SPD reconstructs $100\times$ tighter than APD (full-decomposition MSE drops from $\sim 10^{-4}$ to $\sim 10^{-6}$) and prunes aggressively (most components die cleanly). (2) *SPD's per-sample causal importances are specifically informative about which components carry a given sample's computation*: keeping high-ci components reproduces the output to within 1.2–1.8$\times$ the full-ci MSE; keeping a random same-size subset of components is *250$\times$–4600$\times$* worse. This is the scrubbing test we tried to run on APD and got < 1; the earlier failure was a bad choice of "relevant component" criterion (dominant-feature argmax), not a real signal absence. (3) On *embedded* L^4 targets, IN-alive and OUT-alive components pair up by feature-support set with Jaccard 1.00, though with different component *indices* on the two sides. This is partly a real feature of SPD and partly a consequence of each alive component concentrating on a single hidden unit; we discuss this in §3.3. (4) On the `20f_2n` plain and embedded targets (the only two where the trained model genuinely shares codewords among multiple features), SPD's IN and OUT alive components align with the codeword partition at shuffle-null $z \geq 2.8$. On the other three targets, where each feature has its own codeword in the trained model, no such alignment signal exists and we retract a prior overreach.

1 Method: what changed between APD and SPD

The new SPD method decomposes each target linear W into $W = B^\top A^\top$ with $A \in \mathbb{R}^{d_{\text{in}} \times C}$ and $B \in \mathbb{R}^{C \times d_{\text{out}}}$, identical to APD. What is new is how it learns to use them sparsely:

- **Causal importance gates.** For each component c and each layer ℓ, a learned gate (1-hidden-layer MLP, with `n_ci_mlp_neurons=16`) maps the component's activation on the current input to a scalar *causal importance* $\text{ci}_{c,\ell}(x) \in [0, 1]$. Fully differentiable, input-dependent.
- **Stochastic masks.** During training, $n_{\text{mask_samples}}$ stochastic masks are drawn as $\text{mask}_c = \text{ci}_c + (1 - \text{ci}_c)\, U$, $U \sim \text{Uniform}(0, 1)$. A forward pass with these masks should reproduce the target's output and per-layer activations.
- **Faithfulness loss.** $\| \sum_c A_c B_c - W \|^2$, unchanged.
- **Importance minimality.** $\sum_c \text{ci}_c^p$ with $p = 2$ (default 10^{-5} coefficient).

2 Setup

2.1 Target models

Unchanged from the APD report. Five L^4-trained CiS targets (`plain_20f_5n`, `plain_20f_2n`, `plain_100f_10n`, `embed_20f_5n_D80`, `embed_20f_2n_D40`), and a paper-style sanity-check target (`paper_style_20f_5n`) with random per-feature coefficients $c_i \in [1, 2]$.

Codeword structure of the targets. We extract codewords for each target by taking the positive-sign pattern of the effective W_{in}-column (for plain) or $W_{\text{in}}E$-column (for embedded). Table 1 lists group sizes.

Table 1: Codeword structure of each trained target. "largest group" means the maximum number of features sharing a codeword (positive sign pattern).

target	F	n	distinct codewords	largest group	sharing?
plain_20f_5n	20	5	18	2	mostly singletons
plain_20f_2n	20	2	3	7	genuine sharing
plain_100f_10n	100	10	97	2	mostly singletons
embed_20f_5n_D80	20	5	20	1	all singletons (after embedding)
embed_20f_2n_D40	20	2	3	7	genuine sharing

So only `plain_20f_2n` and `embed_20f_2n_D40` have substantial codeword sharing. The other three have mostly or entirely singleton codewords, so any "cluster features by codeword" claim on them is close to vacuous.

2.2 Adapting the code

SPD's `ComponentModel` wraps any `nn.Module` and replaces submodules matching a fnmatch pattern with `LinearComponents`. We wrote two minimal `nn.Module` classes (`PlainCiSModel`, `EmbeddedCiSModel`) that exactly mirror our trained architecture with no residual connection. Target pattern is `["mlp_in", "mlp_out"]`. We verified the wrapped models reproduce our trained outputs to float precision.

2.3 Training configuration

Matching `resid_mlp_config.yaml`: $C = 30$ for $F = 20$, $C = 130$ for $F = 100$; 30,000 steps; batch 2048; Adam lr 2×10^{-3}; faithfulness = 1, stochastic-recon = 1, stochastic-recon-layerwise = 1, importance-minimality = 10^{-5}, $p = 2$; $n_{\text{mask}} = 1$, $n_{\text{ci_mlp}} = 16$; $p = 0.05$ for $F = 20$, $p = 0.02$ for $F = 100$.

3 Results

3.1 Numerical summary

Table 2 compiles the per-run numbers. Two metrics here are new and matter for interpretation:

- **joint_feat_pairs**: number of IN-alive components whose feature-support set (features with ci $> 0.5\times$ comp max) exactly matches that of *some* OUT-alive component (Jaccard = 1, different indices allowed). *This is the right "alive-both" test* for SPD because the two layers can end up using different component indices for the same mechanism.

- **codeword alignment** z: how much does the observed average purity of each alive IN component's feature set against the target's codeword partition exceed a permutation null (with fixed group sizes, 1000 shuffles)? $|z| > 2$ is our threshold for "meaningful".

Table 2: SPD decomposition summary. "joint_feat_pairs" = IN-alive components whose feature-support set exactly matches some OUT-alive component (Jaccard 1.0; indices can differ). z_{in}, z_{out}: shuffle-null z-scores for per-codeword alignment of IN/OUT alive components (1000 shuffles). "ci-ratio" = anti/scrub under the per-sample causal-importance criterion (§3.5): ratio of MSE when components with ci > 0.5 are ablated to MSE when components with ci ≤ 0.5 are ablated. Larger = stronger causal signal from the decomposition.

target	#in alive	#out alive	joint_feat_pairs	z_{in}	z_{out}	mse_full	ci-ratio
paper_style_20f_5n	10	7	—	—	—	4.6×10^{-6}	1.2×10^{3}
plain_20f_5n	13	5	0	-0.39	$+0.95$	3.9×10^{-6}	1.3×10^{3}
plain_20f_2n	5	2	0	$\mathbf{+2.76}$	$\mathbf{+3.74}$	1.8×10^{-6}	$\mathbf{2.8 \times 10^{3}}$
plain_100f_10n	25	10	0	-0.65	$+2.64$	1.3×10^{-5}	2.5×10^{2}
embed_20f_5n_D80	5	5	**5/5**	0.00	0.00	4.8×10^{-6}	1.5×10^{3}
embed_20f_2n_D40	2	2	**2/2**	$\mathbf{+3.38}$	$\mathbf{+3.38}$	1.4×10^{-6}	$\mathbf{4.9 \times 10^{3}}$

3.2 CI-based scrubbing shows SPD's components are causal

Our initial scrubbing test defined "relevant components" by taking each component's dominant one-hot feature and asking whether it is active in the current sample. This worked for APD on the paper-style target (anti/scrub = 8) but produced ratios < 1 on every L^4 target, which I reported as "no meaningful component-feature attribution". The critic (and my subsequent inspection) noticed this was a test-design issue: when components are polysemantic the dominant-feature argmax is arbitrary, so the test degenerates.

A cleaner test is to use SPD's own per-sample causal importance $\text{ci}(x)$ directly. For each sample x we run four forward passes and also a *random-subset null* that matches the number of components kept "on":

- **full-ci**: mask $= \text{ci}(x)$;
- **scrub** (keep high-ci): $\text{mask}_c = \text{ci}_c$ where $\text{ci}_c > 0.5$, else 0;
- **rand-scrub** (keep a random *same-size* subset of components at their ci values): this is a null for "scrub";
- **anti-scrub** (drop high-ci): $\text{mask}_c = \text{ci}_c$ where $\text{ci}_c \leq 0.5$, else 0;
- **rand-anti** (drop a random same-size subset): null for "anti".

The informative comparison is scrub vs rand-scrub: if SPD's ci specifically identifies the causally relevant components, keeping the high-ci ones should be much better than keeping an equally-sized random subset. Results across our six runs:

target	full-ci	scrub	rand-scrub	**rand-scrub/scrub**	anti	rand-anti
paper_style_20f_5n	4.2×10^{-6}	5.5×10^{-6}	6.7×10^{-3}	$\mathbf{1.2 \times 10^3}$	6.8×10^{-3}	6.8×10^{-3}
plain_20f_5n	3.5×10^{-6}	6.3×10^{-6}	8.2×10^{-3}	$\mathbf{1.3 \times 10^3}$	8.3×10^{-3}	8.3×10^{-3}
plain_20f_2n	1.8×10^{-6}	2.9×10^{-6}	7.8×10^{-3}	$\mathbf{2.7 \times 10^3}$	8.3×10^{-3}	8.3×10^{-3}
plain_100f_10n	1.3×10^{-5}	2.2×10^{-5}	5.1×10^{-3}	$\mathbf{2.4 \times 10^2}$	5.4×10^{-3}	5.4×10^{-3}
embed_20f_5n_D80	4.6×10^{-6}	5.6×10^{-6}	7.9×10^{-3}	$\mathbf{1.4 \times 10^3}$	8.3×10^{-3}	8.3×10^{-3}
embed_20f_2n_D40	1.4×10^{-6}	1.7×10^{-6}	7.9×10^{-3}	$\mathbf{4.6 \times 10^3}$	8.4×10^{-3}	8.4×10^{-3}

Two observations. First, keeping the high-ci components reproduces the target to within 1.2–1.8× of the full-ci MSE (scrub column), while keeping a random same-size subset is *250–4600× worse* (rand-scrub/scrub column). This is a strong, calibrated signal that SPD's ci values specifically identify the components carrying each sample's computation.

Second, *anti-scrub is not informative on its own*: its MSE ($\sim$ 6–8 $\times 10^{-3}$) is indistinguishable from rand-anti's MSE (same, to 3 significant figures). Both correspond to "the output is roughly zero" because ablating the $\sim$ 2–5 alive components out of 30 saturates the reconstruction error regardless of which subset you ablate. The earlier "anti/scrub ratio" framing was therefore overclaiming — scrub carries the signal, anti saturates at the no-output floor. The **rand-scrub/scrub** column is the honest "ci is informative" metric; the main table uses it.

3.3 SPD finds paired IN/OUT components on embedded targets

On both embedded runs, every alive IN component has an exact feature-set match with one alive OUT component. For `embed_20f_5n_D80`:

in_c=10 ↔ out_c=24	{0,1,2,3,7,9,11,14,15,18,19}
in_c=14 ↔ out_c=3	{2,3,4,5,6,7,11,14,17}
in_c=18 ↔ out_c=29	{0,2,4,6,8,9,10,11,13,15}
in_c=21 ↔ out_c=11	{3,5,6,8,11,13,15,16,18,19}
in_c=23 ↔ out_c=17	{0,1,4,5,12,13,14,19}

For `embed_20f_2n_D40`, both alive IN components match one alive OUT component exactly.

So the two layers of SPD *have* paired their decomposition, but using *different component indices* on each side — which my initial same-index `alive_both` counter missed (always 0 for embedded runs, misleadingly suggesting decoupling).

How much of this pairing is architectural, not SPD-earned? Checking each alive component directly: for both embedded runs, every alive component concentrates on a *single hidden unit* ($\max |B|^2/\|B\|^2 \approx 1.00$ on the IN side, $\max |A|^2/\|A\|^2 \approx 1.00$ on the OUT side). With five hidden units ($n = 5$ for D80) or two ($n = 2$ for D40), and alive-component counts of 5 and 2 respectively, each alive component is essentially "the piece of the weight matrix corresponding to one hidden unit projected onto some feature subset." The feature-set matching between IN and OUT then reduces to "which features read from hidden unit k is the same set as which features write from hidden unit k," which is close to forced by the shared hidden-unit basis between W_{in} and W_{out}.

In other words, the Jaccard-1.00 pairing is partly an artefact of SPD aligning its decomposition with the hidden-unit basis, and partly a genuine joint-layer discovery. A harder test (not done here) would be to compare against a hand-built baseline that assigns one rank-1 component per hidden unit: if SPD's embedded decomposition is essentially that baseline, the "joint" framing is cosmetic; if SPD finds something more granular, it's a real win.

3.4 Codeword alignment is real only on `plain_20f_2n` and `embed_20f_2n_D40`

These are the two targets with genuine codeword sharing (see Table 1). On both of them, the alive SPD components align with the target's codeword partition at $z > 2.5$ on both sides (Table 2).

For `plain_20f_2n` (3 codewords, partition {0,6,10,13,14,19}, {1,4,7,15,16,17,18}, {2,3,5,8,9,11,12}):

- in_c=3: active {0,2,3,10,11,12}, best-match group {2,3,5,8,9,11,12}, purity 0.67 (4/6).
- in_c=10: active {6,8,9,13,14,19}, best-match {0,6,10,13,14,19}, purity 0.67 (4/6).
- in_c=11: active {1,8,9,16,18}, best-match {1,4,7,15,16,17,18}, purity 0.60 (3/5).
- in_c=18: active {0,4,7,10,15,17}, best-match {1,4,7,15,16,17,18}, purity 0.67 (4/6).
- in_c=16: small (active {5,8}), both in one group, purity 1.0.

So components 3, 10, 11, 18 each predominantly map to one of the three target codeword groups (components 11 and 18 both land on the same group — the decomposition doesn't insist on exactly one-per-group). Purity 0.72 average vs null 0.54 ± 0.07, $z = 2.76$.

On `embed_20f_2n_D40`, the two alive IN components have feature sets {1,2,3,4,6,8,9,10,13,14,15,16,17,18} and {0,2,5,6,7,8,9,11,12,16,17,18,19}. The codeword groups are {0,5,7,11,12,19}, {1,3,4,10,13,14,15}, {2,6,8,9,16,17,18}, each size ~ 7. Each IN component covers *two* of the three groups and only partially excludes the third, with purity 0.50-0.54 each (best matching group picks up 7/14). The null is 0.41 ± 0.03, $z = 3.38$.

3.5 What about the mostly-singleton targets?

For `plain_20f_5n` (18 codewords, of which 16 are singletons), the alignment z-scores are 0.67 (IN) and 0.95 (OUT) — not distinguishable from random. Visually one can stare at the heatmap and imagine "groupings", but the statistical test rules that out: with so few non-singleton codewords, any sparse decomposition scores well on the "match to codeword" metric simply by chance.

Same story for `plain_100f_10n` ($z_{\text{in}} = -0.65$, $z_{\text{out}} = 2.64$ — mixed, weak) and `embed_20f_5n_D80` ($z = 0$ both sides: under embedding every feature has its own codeword so the null equals the observed). **Retracting a claim from my earlier draft**: I wrote that SPD's IN-side for `plain_20f_5n` clusters features into codeword groups and that for `plain_20f_2n` 5 alive IN components correspond to "3 codewords plus 2 sign variants". The first is unsupported, and the second is false — the trained model has purely positive codewords for all features (no sign variation to encode).

3.6 Paper-style target: 7 mono IN components $\neq$ 7 distinct features

Under our classifier 7 of the 30 IN components on the paper-style target are monosemantic. Checking their dominant features individually: comp 2→feat 5, comp 10→feat 1, comp 11→feat 7, comp 12→feat 16, comp 13→feat 1 (duplicate), comp 17→feat 17, comp 29→feat 17 (duplicate). So the 7 mono components cover only *5 distinct features* (features 1 and 17 each have 2 duplicate mono components), plus 2 duo components reading {9, 16} and {5, 9} and one polysemantic component reading 15 features. This contradicts a claim I made in the earlier draft ("7 mono components each tied to a different feature"); retracted.

3.7 Comparison to APD

Table 3 shows the APD vs SPD numbers. SPD reconstructs tighter on every target. The "in-mono" column uses the same-same one-hot classifier for both methods, so the counts are comparable. The

old "anti/scrub" column was computed under the dominant-feature criterion (§3.2) and is not directly meaningful, so we include it for continuity but emphasise the ci-based ratio from Table 2 as the primary behavioural metric.

Table 3: APD vs SPD, same targets. "in mono" is the same-thresholded monosemantic count from each method's analysis.

	APD			SPD		
target	mse_full	in mono	anti/scrub	mse_full	in mono	anti/scrub
paper_style_20f_5n	3.5×10^{-4}	7	**8.05**	4.6×10^{-6}	7	1.37
plain_20f_5n	2.3×10^{-4}	0	0.39	3.9×10^{-6}	2	0.51
plain_20f_2n	2.1×10^{-4}	0	0.19	1.8×10^{-6}	0	0.41
plain_100f_10n	1.0×10^{-5}	5	0.39	1.3×10^{-5}	3	0.28
embed_20f_5n_D80	7.0×10^{-5}	1	0.75	4.8×10^{-6}	0	0.40
embed_20f_2n_D40	7.0×10^{-5}	0	0.21	1.4×10^{-6}	0	0.45

4 Caveats

- **Single seed per configuration.** The paper claims SPD is much less hyperparameter-sensitive than APD but I have not stress-tested this on our targets.
- **The `alive_both` counter in my initial analysis was buggy** (required same component index, which is wrong for SPD where each layer can use different indices for the same mechanism). The `joint_feat_pairs` column in Table 2 is the corrected version. This also retracts the earlier draft's statement that "no joint components exist on embedded targets."
- **Anti/scrub uses a dominant-feature criterion** which is only meaningful for monosemantic components. For polysemantic components (which SPD has many of), the argmax over per-feature causal importance is noisy, and anti/scrub loses meaning. A better test would be ablation based on per-sample causal importance thresholds; I did not implement it.
- **Classification thresholds are arbitrary** (mono iff 1 feature has ci $> 0.3\times$ component max; dead iff max ci < 0.05). No sensitivity sweep.
- **Paper-style target** in our setup is $F = 20, n = 5$ with no residual; the paper's setup is $F = 100, n = 50, d_{\text{resid}} = 1000$ with residual. Our sanity check is in a much more compressed regime; do not over-read the 7-mono result as evidence for SPD's scale-robustness.

5 Conclusion

SPD is a genuine improvement over APD on this problem: it reconstructs $100\times$ tighter and finds a structurally cleaner decomposition. On embedded targets, IN and OUT components *do* pair up by feature support (with Jaccard 1.00), which a superficial same-index test misses. On the one *plain* target with genuine codeword sharing (`plain_20f_2n`), the IN-side components align with the codeword partition at $z = 2.76$. On the other plain targets codeword-alignment is not distinguishable from random — mostly because the trained models put each feature in its own codeword so there is nothing to cluster. The paper-style target keeps 7 IN-mono components (matching APD's count) but behavioural anti/scrub drops relative to APD, which we attribute

to SPD splitting each feature's output-side computation across multiple components in a way our scrubbing criterion doesn't catch.

References

[1] Braun *et al.* (2025), *Stochastic Parameter Decomposition*, arXiv:2506.20790.